

```

# Environment Setup and Dependencies
print("=" * 80)
print("SINQ Quantization for Qwen/Qwen3-8B")
print("=" * 80)

# Check GPU availability
import subprocess
import sys
import os

def check_gpu():
    """Check if GPU is available and print GPU info"""
    try:
        gpu_info = subprocess.check_output(['nvidia-smi'], stderr=subprocess.STDOUT).decode()
        print("GPU Information:")
        print("-" * 40)
        for line in gpu_info.split('\n'):
            if 'NVIDIA' in line or 'Memory' in line:
                print(line)
        return True
    except:
        print("No GPU detected! SINQ quantization requires GPU.")
        return False

if not check_gpu():
    raise RuntimeError("This script requires GPU. Please enable GPU in Colab: Runtime > Change runtime type > GP

print("\n" + "=" * 80)
print("Step 1: Installing Dependencies")
print("=" * 80)

# Install latest transformers from source
!pip install -q git+https://github.com/huggingface/transformers.git
!pip install -q torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cu118
!pip install -q accelerate bitsandbytes
!pip install -q sentencepiece protobuf
!pip install -q datasets

# Clone and install SINQ repository
print("\n Cloning SINQ repository...")
!git clone https://github.com/huawei-csl/SINQ.git
%cd SINQ

# Install SINQ dependencies
print("Installing SINQ requirements...")
!pip install -q -r req.txt
!pip install -q scipy --upgrade
!pip install -q triton --upgrade
!pip install -q .

# Import necessary libraries
import torch
import gc
import json
import time
from datetime import datetime
from pathlib import Path
import numpy as np
from transformers import AutoTokenizer, AutoModelForCausalLM, AutoConfig
from sinq.patch_model import AutoSINQHFModel
from sinq.sinqlinear import BaseQuantizeConfig
from datasets import load_dataset
import warnings
warnings.filterwarnings('ignore')

```

```

=====
SINQ Quantization for Qwen/Qwen3-8B
=====

```

```

GPU Information:

```

```

=====
| NVIDIA-SMI 550.54.15      Driver Version: 550.54.15      CUDA Version: 12.4      |
| Fan  Temp  Perf    Pwr:Usage/Cap |      Memory-Usage | GPU-Util  Compute M. |
|   0  NVIDIA A100-SXM4-80GB     Off |  00000000:00:05.0 Off |                    0 |
| GPU   GI   CI          PID    Type   Process name          GPU Memory |
=====

```

```

Step 1: Installing Dependencies

```

```

Installing build dependencies ... done
Getting requirements to build wheel ... done
Preparing metadata (pyproject.toml) ... done

```



```

===== 528.7/528.7 kB 36.9 MB/s eta 0:00:00
===== 46.8/46.8 kB 4.3 MB/s eta 0:00:00
Building wheel for transformers (pyproject.toml) ... done
===== 60.1/60.1 MB 44.6 MB/s eta 0:00:00

Cloning SINQ repository...
Cloning into 'SINQ'...
remote: Enumerating objects: 200, done.
remote: Counting objects: 100% (98/98), done.
remote: Compressing objects: 100% (17/17), done.
remote: Total 200 (delta 85), reused 86 (delta 81), pack-reused 102 (from 1)
Receiving objects: 100% (200/200), 1.35 MiB | 1.48 MiB/s, done.
Resolving deltas: 100% (108/108), done.
/content/SINQ
Installing SINQ requirements...
===== 60.6/60.6 kB 6.1 MB/s eta 0:00:00
===== 50.5/50.5 kB 4.7 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
===== 51.8/51.8 kB 4.9 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
Preparing metadata (setup.py) ... done
===== 561.5/561.5 kB 41.8 MB/s eta 0:00:00
===== 38.2/38.2 MB 73.7 MB/s eta 0:00:00
===== 11.3/11.3 MB 140.1 MB/s eta 0:00:00
===== 3.9/3.9 MB 111.2 MB/s eta 0:00:00
===== 84.1/84.1 kB 8.5 MB/s eta 0:00:00
===== 293.6/293.6 kB 31.5 MB/s eta 0:00:00
===== 104.1/104.1 kB 11.8 MB/s eta 0:00:00
===== 3.1/3.1 MB 117.7 MB/s eta 0:00:00
===== 91.1/91.1 kB 10.2 MB/s eta 0:00:00
Building wheel for rouge-score (setup.py) ... done
Building wheel for sqllitedict (setup.py) ... done
Building wheel for word2number (setup.py) ... done
ERROR: pip's dependency resolver does not currently take into account all the packages that are installed. This b
tsfresh 0.21.1 requires scipy>=1.14.0; python_version >= "3.10", but you have scipy 1.13.1 which is incompatible.
===== 62.0/62.0 kB 5.4 MB/s eta 0:00:00
===== 35.7/35.7 MB 68.9 MB/s eta 0:00:00
Preparing metadata (setup.py) ... done
Building wheel for sinq (setup.py) ... done

```

```

# Configuration and Setup
print("\n" + "=" * 80)
print("Step 2: Configuration Setup")
print("=" * 80)

class QuantizationConfig:
    """Configuration class for SINQ quantization parameters"""

    def __init__(self):
        self.model_name = "Qwen/Qwen3-8B"
        self.cache_dir = "/content/model_cache"
        self.nbits = 4
        self.group_size = 128
        self.tiling_mode = "1D"
        self.method = "sinq"
        self.compute_dtype = torch.bfloat16
        self.max_memory = {0: "70GB"}
        self.offload_folder = "/content/offload"
        self.output_dir = "/content/quantized_models"
        self.save_quantized = True
        self.run_evaluation = True
        self.eval_task = "lambada_openai"
        self.eval_limit = 100 # Limit to 100 samples for speed

    def to_dict(self):
        return {
            "model_name": self.model_name,
            "nbits": self.nbits,
            "group_size": self.group_size,
            "tiling_mode": self.tiling_mode,
            "method": self.method,
            "compute_dtype": str(self.compute_dtype),
            "timestamp": datetime.now().isoformat()
        }

config = QuantizationConfig()

Path(config.cache_dir).mkdir(parents=True, exist_ok=True)
Path(config.output_dir).mkdir(parents=True, exist_ok=True)
Path(config.offload_folder).mkdir(parents=True, exist_ok=True)

print(f" Configuration initialized:")
print(f"   - Model: {config.model_name}")

```

```

print(f" - Quantization: {config.nbits}-bit, group_size={config.group_size}")
print(f" - Method: {config.method}")
print(f" - Tiling: {config.tiling_mode}")
print(f" - Note: Using A100 (80GB). If OOM occurs, restart runtime with high-RAM.")

```

Step 2: Configuration Setup

```

=====
Configuration initialized:
- Model: Qwen/Qwen3-8B
- Quantization: 4-bit, group_size=128
- Method: sinq
- Tiling: 1D
- Note: Using A100 (80GB). If OOM occurs, restart runtime with high-RAM.
=====

```

```

# Memory Management Utilities
print("\n" + "=" * 80)
print("Step 3: Setting up Memory Management")
print("=" * 80)

def clear_gpu_memory():
    """Clear GPU memory cache"""
    gc.collect()
    torch.cuda.empty_cache()
    if torch.cuda.is_available():
        torch.cuda.synchronize()

def print_memory_usage():
    """Print current GPU memory usage"""
    if torch.cuda.is_available():
        allocated = torch.cuda.memory_allocated(0) / 1024**3
        reserved = torch.cuda.memory_reserved(0) / 1024**3
        print(f"GPU Memory: Allocated={allocated:.2f}GB, Reserved={reserved:.2f}GB")

clear_gpu_memory()
print_memory_usage()

```

Step 3: Setting up Memory Management

```

=====
GPU Memory: Allocated=0.00GB, Reserved=0.00GB
=====

```

```

# Model Loading with Optimization
print("\n" + "=" * 80)
print("Step 4: Loading Model and Tokenizer")
print("=" * 80)

def load_model_optimized(config):
    """Load model with memory optimization"""
    print(f"Loading tokenizer from {config.model_name}...")
    try:
        tokenizer = AutoTokenizer.from_pretrained(
            config.model_name,
            cache_dir=config.cache_dir,
            trust_remote_code=True
        )
    except Exception as e:
        print(f"Failed to load tokenizer: {e}")
        raise

    if tokenizer.pad_token is None:
        tokenizer.pad_token = tokenizer.eos_token
        print(" - Set pad_token to eos_token")

    print(f"Loading model configuration...")
    try:
        model_config = AutoConfig.from_pretrained(
            config.model_name,
            cache_dir=config.cache_dir,
            trust_remote_code=True
        )
    except Exception as e:
        print(f"Failed to load model config: {e}")
        raise

    print(f" Loading model with optimizations...")
    print(" This may take a few minutes for an 8B model...")

```

```
clear_gpu_memory()

try:
    model = AutoModelForCausalLM.from_pretrained(
        config.model_name,
        cache_dir=config.cache_dir,
        torch_dtype=config.compute_dtype,
        device_map="cuda:0",
        max_memory=config.max_memory,
        trust_remote_code=True,
        low_cpu_mem_usage=True
    )
    model = model.to_empty(device="cuda:0") if model.device.type == "meta" else model.to("cuda:0")
    print("Model loaded successfully on GPU")
except Exception as e:
    print(f"Failed to load model: {e}")
    print("Try restarting runtime or reducing model size.")
    raise

total_params = sum(p.numel() for p in model.parameters())
print(f"\nModel Statistics:")
print(f"  - Total parameters: {total_params / 1e9:.2f}B")
print(f"  - Model dtype: {next(model.parameters()).dtype}")

print_memory_usage()

return model, tokenizer

print("\n Starting model loading process...")
start_time = time.time()

try:
    model, tokenizer = load_model_optimized(config)
    load_time = time.time() - start_time
    print(f"\nModel loaded in {load_time:.2f} seconds")
except Exception as e:
    print(f"Model loading failed: {e}")
    print("Try restarting runtime or using a smaller model.")
    sys.exit(1)

clear_gpu_memory()
```

```
=====
Step 4: Loading Model and Tokenizer
=====
```

```
Starting model loading process...
Loading tokenizer from Qwen/Qwen3-8B...
tokenizer_config.json: 9.73k/? [00:00<00:00, 1.05MB/s]

vocab.json: 2.78M/? [00:00<00:00, 98.6MB/s]

merges.txt: 1.67M/? [00:00<00:00, 84.1MB/s]

tokenizer.json: 100% 11.4M/11.4M [00:01<00:00, 2.45MB/s]
Loading model configuration...
config.json: 100% 728/728 [00:00<00:00, 91.5kB/s]
Loading model with optimizations...
This may take a few minutes for an 8B model...
model.safetensors.index.json: 32.9k/? [00:00<00:00, 3.87MB/s]

Fetching 5 files: 100% 5/5 [01:25<00:00, 35.39s/it]

model-00001-of-00005.safetensors: 100% 4.00G/4.00G [01:14<00:00, 74.4MB/s]
model-00005-of-00005.safetensors: 100% 1.24G/1.24G [00:17<00:00, 101MB/s]
model-00003-of-00005.safetensors: 100% 3.96G/3.96G [01:24<00:00, 46.6MB/s]
model-00004-of-00005.safetensors: 100% 3.19G/3.19G [01:23<00:00, 19.3MB/s]
model-00002-of-00005.safetensors: 100% 3.99G/3.99G [01:24<00:00, 547MB/s]

Loading checkpoint shards: 100% 5/5 [00:04<00:00, 1.18it/s]
generation_config.json: 100% 239/239 [00:00<00:00, 26.9kB/s]
Model loaded successfully on GPU

Model Statistics:
- Total parameters: 8.19B
- Model dtype: torch.bfloat16
GPU Memory: Allocated=15.26GB, Reserved=15.26GB

Model loaded in 106.09 seconds
```

```
# SINQ Quantization Implementation
print("\n" + "=" * 80)
print("Step 5: Applying SINQ Quantization")
print("=" * 80)

def quantize_with_sinq(model, tokenizer, config):
    """Apply SINQ quantization to the model"""
    print(f"\n Configuring SINQ quantization...")

    quant_cfg = BaseQuantizeConfig(
        nbits=config.nbits,
        group_size=config.group_size,
        tiling_mode=config.tiling_mode,
        method=config.method
    )

    print(f" - Bits: {config.nbits}")
    print(f" - Group size: {config.group_size}")
    print(f" - Tiling mode: {config.tiling_mode}")
    print(f" - Method: {config.method}")

    print(f"\n Starting SINQ quantization...")
    print(" This process may take 5-15 minutes for an 8B model...")

    start_time = time.time()

    try:
        AutoSINQHFModel.quantize_model(
            model,
            tokenizer=tokenizer,
            quant_config=quant_cfg,
            compute_dtype=config.compute_dtype,
            device="cuda:0"
        )

        quant_time = time.time() - start_time
        print(f"\n Quantization completed in {quant_time:.2f} seconds")
```

```

print(f"\n Quantization Results:")
print(f"   - Original precision: 16-bit")
print(f"   - Quantized precision: {config.nbits}-bit")
print(f"   - Theoretical compression ratio: {16 / config.nbits:.2f}x")
print(f"   - Expected memory reduction: ~{(1 - config.nbits/16) * 100:.1f}%")

return True

except Exception as e:
    print(f"\nQuantization failed: {e}")
    import traceback
    traceback.print_exc()
    return False

print("\n Initiating SINQ quantization process...")
quantization_success = quantize_with_sinq(model, tokenizer, config)

if not quantization_success:
    print("\n Quantization failed. Please check the error messages above.")
    sys.exit(1)

print_memory_usage()
clear_gpu_memory()

```

===== Step 5: Applying SINQ Quantization =====

```

Initiating SINQ quantization process...

Configuring SINQ quantization...
- Bits: 4
- Group size: 128
- Tiling mode: 1D
- Method: sinq

< Starting SINQ quantization...
This process may take 5-15 minutes for an 8B model...
100%|██████████| 183/183 [00:00<00:00, 7021.71it/s]
100%|██████████| 253/253 [00:06<00:00, 39.43it/s]

Quantization completed in 7.58 seconds

Quantization Results:
- Original precision: 16-bit
- Quantized precision: 4-bit
- Theoretical compression ratio: 4.00x
- Expected memory reduction: ~75.0%
GPU Memory: Allocated=5.76GB, Reserved=15.61GB

```

```

# Model Saving
if config.save_quantized:
    print("\n" + "=" * 80)
    print("Step 6: Saving Quantized Model with Reloadable Weights")
    print("=" * 80)

    save_path = Path(config.output_dir) / f"qwen3_8b_{config.nbits}bit_{config.method}"
    save_path.mkdir(parents=True, exist_ok=True)

    print(f" Saving quantized model to {save_path}...")

    try:
        # Save the quantized model weights
        AutoSINQHfModel.save_quantized(model, str(save_path), verbose=True)

        # Save tokenizer
        tokenizer.save_pretrained(save_path)

        # Save configuration
        config_path = save_path / "quantization_config.json"
        with open(config_path, 'w') as f:
            json.dump(config.to_dict(), f, indent=2)

        # Optional: Save the model using PyTorch's state_dict for direct loading
        torch.save(model.state_dict(), save_path / "pytorch_model.bin")

        total_size = sum(f.stat().st_size for f in save_path.glob("**/*") if f.is_file())
        print(f"\n Model saved successfully!")
        print(f"   - Save location: {save_path}")
        print(f"   - Total size: {total_size / 1e9:.2f} GB")
    
```

```

print(f"\n You can reload this quantized model directly with:\n"
      f"   from sinq.patch_model import AutoSINQHModel\n"
      f"   model = AutoSINQHModel.from_pretrained('{save_path}'))

except Exception as e:
    print(f" Failed to save model: {e}")

```

Step 6: Saving Quantized Model with Reloadable Weights

Saving quantized model to /content/quantized_models/qwen3_8b_4bit_sinq...

Model saved successfully!

- Save location: /content/quantized_models/qwen3_8b_4bit_sinq
- Total size: 8.69 GB

You can reload this quantized model directly with:

```

from sinq.patch_model import AutoSINQHModel
model = AutoSINQHModel.from_pretrained('/content/quantized_models/qwen3_8b_4bit_sinq')

```

Fast Model Evaluation

```

if config.run_evaluation:
    print("\n" + "=" * 80)
    print("Step 7: Fast Model Evaluation")
    print("=" * 80)

    def evaluate_lambda(model, tokenizer, task, limit, batch_size=4):
        """Lightweight evaluation on a subset of lambada_openai"""
        print(f"Running fast evaluation on {task} (limited to {limit} samples)...")
        try:
            dataset = load_dataset("lambada", split="test")[:limit]
            correct = 0
            total = 0

            # Process in batches
            for i in range(0, len(dataset), batch_size):
                batch = dataset[i:i+batch_size]
                inputs = tokenizer(
                    batch["text"],
                    return_tensors="pt",
                    padding=True,
                    truncation=True,
                    max_length=512
                ).to("cuda:0")

                with torch.no_grad():
                    outputs = model.generate(
                        *inputs,
                        max_new_tokens=1,
                        pad_token_id=tokenizer.eos_token_id
                    )

                for j, output in enumerate(outputs):
                    predicted = tokenizer.decode(output[input_ids][j].size(0):, skip_special_tokens=True)
                    target = batch["text"][j].split()[-1]
                    if predicted.lower() == target.lower():
                        correct += 1
                    total += 1

            clear_gpu_memory()

            accuracy = correct / total if total > 0 else 0
            print(f"\n Evaluation Results for {task}:")
            print(f"   - Accuracy: {accuracy:.4f} ({correct}/{total} correct)")

        except Exception as e:
            print(f" Evaluation failed: {e}")

    try:
        evaluate_lambda(model, tokenizer, config.eval_task, config.eval_limit, batch_size=4)
    except Exception as e:
        print(f" Fast evaluation failed: {e}")

```

```
=====
Step 7: Fast Model Evaluation
=====
```

Running fast evaluation on lambada_openai (limited to 100 samples)...

```
README.md:      7.32k/? [00:00<00:00, 805kB/s]

plain_text/train-00000-of-00002.parquet: 100%          269M/269M [00:08<00:00, 29.2MB/s]
plain_text/train-00001-of-00002.parquet: 100%          281M/281M [00:01<00:00, 337MB/s]
plain_text/test-00000-of-00001.parquet: 100%           1.14M/1.14M [00:00<00:00, 1.66MB/s]
plain_text/validation-00000-of-00001.par(...): 100%     1.08M/1.08M [00:00<00:00, 1.89MB/s]

Generating train split: 100%                          2662/2662 [00:05<00:00, 430.17 examples/s]
Generating test split: 100%                           5153/5153 [00:00<00:00, 256467.06 examples/s]
Generating validation split: 100%                      4869/4869 [00:00<00:00, 266203.48 examples/s]

Evaluation failed: slice(0, 4, None)
```

```
# Inference Testing
print("\n" + "=" * 80)
print("Step 8: Testing Quantized Model Inference")
print("=" * 80)

def test_inference(model, tokenizer, prompts=None):
    """Test the quantized model with sample prompts"""
    if prompts is None:
        prompts = [
            "Describe the future of artificial intelligence.",
            "Explain how quantum computing could impact technology.",
            "What is the most significant technological advancement of the 21st century?",
            "Do you think AI is Dangeours?"
        ]

    print("Testing inference with sample prompts...\n")

    for i, prompt in enumerate(prompts, 1):
        print(f"Prompt {i}: {prompt}")

        inputs = tokenizer(prompt, return_tensors="pt").to("cuda:0")

        with torch.no_grad():
            outputs = model.generate(
                **inputs,
                max_new_tokens=50,
                temperature=0.7,
                do_sample=True,
                top_p=0.9,
                pad_token_id=tokenizer.eos_token_id
            )

        response = tokenizer.decode(outputs[0], skip_special_tokens=True)
        print(f"Response: {response[len(prompt):]}\n")
        print("-" * 60)

        clear_gpu_memory()

    try:
        test_inference(model, tokenizer)
    except Exception as e:
        print(f" Inference test failed: {e}")
```

```
=====
Step 8: Testing Quantized Model Inference
=====
```

Testing inference with sample prompts...

Prompt 1: Describe the future of artificial intelligence.

Response: The future of artificial intelligence is a multifaceted and rapidly evolving landscape, shaped by tech

```
### 1. **Technological Advancements**
- **General Artificial Intelligence (AG
```

Prompt 2: Explain how quantum computing could impact technology.

Response: Quantum computing has the potential to revolutionize various fields of technology by solving complex p

```
1. **Cryptography**: Quantum computers can break many of the encryption algorithms
```

 Prompt 3: What is the most significant technological advancement of the 21st century?
 Response: How has it impacted the world, and what are the potential risks associated with it?

The most significant technological advancement of the 21st century is **artificial intelligence (AI)**. Here's a
 ### **

 Prompt 4: Do you think AI is Dangerous?
 Response: Or not? What's your opinion?

The question of whether AI is dangerous or not is complex and depends on various factors, including how it's deve
 ### **Potential Risks of AI:

 # Summary
 print("\n" + "=" * 80)
 print("Quantization Complete - Summary")
 print("=" * 80)

 print(f"""
 📊 Quantization Summary:
 - Model: {config.model_name}
 - Quantization Method: {config.method.upper()}
 - Bit-width: {config.nbits}-bit
 - Group Size: {config.group_size}
 - Tiling Mode: {config.tiling_mode}
 - Estimated Memory Reduction: ~{(1 - config.nbits/16) * 100:.1f}%
 """)

 if config.save_quantized:
 print(f"Saved Location: {config.output_dir}")

 print("\n All operations completed successfully!")
 print("\n")